

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana

w specjalności TO-DO

Bot Bowl - nauka sieci neuronowej gry w grę Blood Bowl

inż. Mikołaj Klębowski

numer albumu 299253

promotor

dr inż. Krzysztof Hryniów

WARSZAWA 2025

Bot Bowl - nauka sieci neuronowej gry w grę Blood Bowl

Streszczenie

To jest streszczenie. To jest trochę za krótkie, jako że powinno zająć całą stronę.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Słowa kluczowe: A, B, C

Bot Bowl - learning the neural network game of Blood Bowl

Abstract

This is abstract. This one is a little too short as it should occupy the whole page.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Keywords: X, Y, Z

Spis treści

1	Wstęp	9
2	Sieci Neuronowe	11
2.1	Funkcja aktywacji	12
2.2	Architektura sieci neuronowej	14
2.3	Techniki uczenia sieci neuronowych	16
3	Sieci Konwolucyjne	23
4	Sieci Rekurencyjne	25
5	Sieci LSTM GRU	27
6	Zanik Gradientu/Wybuch gradientu	29
	Bibliografia	31
	Wykaz skrótów i symboli	31
	Spis rysunków	33
	Spis tabel	35

Rozdział 1

Wstęp

W ciągu ostatnich lat sztuczna inteligencja przekształciła wiele obszarów gospodarki i życia codziennego. Systemy oparte na sieciach neuronowych wspierają wyszukiwarki internetowe, umożliwiają autonomiczną jazdę samochodów i optymalizują procesy produkcyjne. Rosnąca zdolność tych modeli do analizowania ogromnych zbiorów danych oraz uczenia się z nich pozwala na podejmowanie decyzji bez bezpośredniego nadzoru człowieka. Przykładowo, w sektorze finansowym sieci neuronowe zarządzają portfelem inwestycyjnym, prognozowanie ryzyka kredytowego czy dynamiczne kształtowanie cen.

Jednak aby w pełni ocenić możliwości i ograniczenia poszczególnych architektur sieci, niezbędne są środowiska testowe o jasno zdefiniowanych regułach, dużej złożoności stanów oraz szerokim wachlarzu możliwych działań. Gry od lat są używane jako środowisko do testowania algorytmów uczenia maszynowego. Dzięki mierzalnym wynikom rozgrywek oraz możliwości powtarzalnego odtwarzania scenariuszy, można precyzyjnie porównywać efektywność różnych metod – od prostych wielowarstwowych perceptronów, przez konwolucyjne sieci neuronowe, aż po nowoczesne modele hybrydowe.

W ramach tej pracy zostanie przeprowadzone porównanie różnych rozwiązań w grze bloodbowl. Ze względu na wysoki branching factor, losowość rzutów kośćmi oraz wieloetapowy charakter rozgrywki, Bloodbowl stanowi doskonałe środowisko do badania procesów decyzyjnych agentów uczonych ze wzmocnieniem. Celem niniejszej pracy jest porównanie wybranych architektur sieci neuronowych pod kątem szybkości konwergencji, stabilności uczenia oraz jakości wypracowanych strategii. W kolejnych rozdziałach przedstawione zostaną szczegóły środowiska BotBowl, opis testowanych modeli oraz wyniki przeprowadzonych eksperymentów.

Rozdział 2

Sieci Neuronowe

Idea sztucznych sieci neuronowych narodziła się podczas prób modelowaniu pracy ludzkiego mózgu. Warren S. McCulloch oraz Walter Pitts w swojej pracy "A logical calculus of the ideas immanent in nervous activity" [McCulloch1943] zaproponowali model sieci neuronowej, jako grafu skierowanego, gdzie węzły to matematyczne modele neuronu takie, że

$$N_i(t+1) = H\left(\sum_{j=1}^n w_{ij}(t)N_j(t) - \Theta_i(t)\right)$$

gdzie:

- N - stan neuronu, może przyjąć wartość 0 lub 1
- H - funkcja skokowa Heaviside'a
- w_{ij} - waga połączenia między wektorem j oraz i .
- Θ - próg wyzwalań, neuron aktywuje się, gdy sygnał wejściowy go przekroczy.

Sieć złożona z takich neuronów jest nazywana dzisiaj perceptronem. Służy ona do zadań klasyfikacji binarnej. Potrafi określić przynależność badanego obiektu do jednej z klas na podstawie wybranych cech. W 1957 roku perceptron stworzony przez Franka Rosenblatt'a został pierwszą uczącą się siecią neuronową. Chociaż perceptron był przełomowy, miał istotne ograniczenie - nie był w stanie rozwiązać problemów nieliniowych, takich jak funkcja XOR. dopiero zastąpienie funkcji skokowej ciągłymi, różniczkowalnymi funkcjami aktywacji umożliwiło trenowanie modeli za pomocą metod opartych o pochodne, takich jak propagacja wsteczna. [minsky1969perceptrons]

Jak zostało wcześniej wspomniane ten model został stworzony w celu opisanie sposobu działania biologicznego neuronu. Można go uogólnić, pozwalając stworzyć bardziej uniwersalny model matematyczny.

$$y = f\left(\sum_{i=1}^m w_i(t)x_i(t) + b\right)$$

gdzie:

- w_i – waga
- x_i – wejście,
- f – funkcja aktywacji,
- y – wyjście,
- m – liczba wejść,
- b – stała wartość (ang. bias).

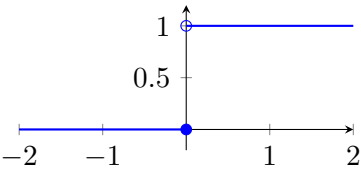
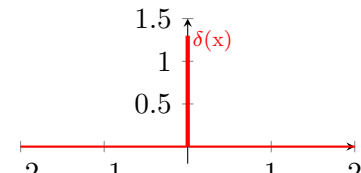
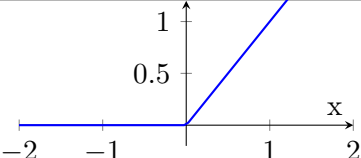
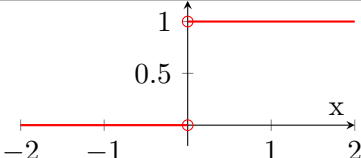
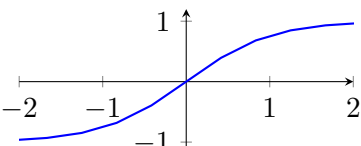
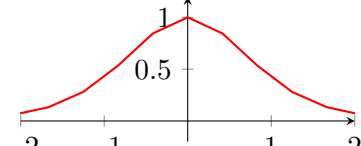
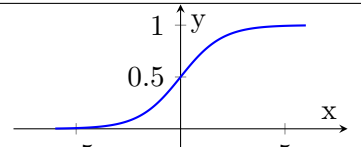
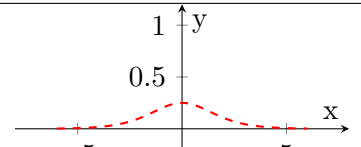
Do funkcji aktywacji f przekazywana jest ważona suma sygnałów wejściowych, do której dodaje się opcjonalnie parametr bias. Taki opis umożliwia zastosowanie dowolnej funkcji aktywacji, co pozwala sieci neuronowej modelować nieliniowe zależności.

2.1 Funkcja aktywacji

Obecnie rzadko stosuje się funkcję skokową Heaviside'a, która była używana w pierwszych modelach sztucznych neuronów. Współczesne sieci opierają się na ciągłych funkcjach aktywacji, takich jak tangens hiperboliczny ($\tanh$) lub funkcja ReLU (Rectified Linear Unit), często z modyfikacjami (np. Leaky ReLU, Parametric ReLU). Ich najważniejszą cechą – z punktu widzenia procesu uczenia – jest różniczkowalność (lub jej wystarczająca przybliżona forma). Właśnie ta właściwość umożliwia zastosowanie algorytmów opartych na obliczaniu gradientu, takich jak propagacja wsteczna błędów.

Na poniższych wykresach przedstawiono porównanie trzech funkcji aktywacji oraz ich pochodnych. Widać wyraźnie, że pochodna funkcji skokowej Heaviside'a jest zerowa wszędzie poza jednym punktem, gdzie dodatkowo nie jest określona. To oznacza, że nie generuje żadnego sygnału uczącego, a tym samym nie nadaje się do wykorzystania w trenowanych sieciach neuronowych. Z kolei funkcja $\tanh(x)$ ma ciągłą i gładką pochodną, która zapewnia niezerowy gradient w szerokim zakresie wejść, co umożliwia stopniowe dostosowywanie wag podczas uczenia. Jednak dla dużych wartości bezwzględnych wejścia gradient zaczyna szybko zanikać, co może utrudniać efektywne trenowanie głębokich sieci.

Funkcja ReLU jest prosta i obliczeniowo efektywna, a jej pochodna jest stała (1) dla dodatnich argumentów i zerowa dla ujemnych. Choć formalnie nie jest różniczkowalna w punkcie 0, w praktyce jej wykorzystanie w uczeniu sieci sprawdza się znakomicie – zwłaszcza w głębokich modelach – ponieważ nie cierpi na zanikający gradient w dodatnim zakresie, co czyni ją funkcją preferowaną w wielu współczesnych zastosowaniach. Wadą ReLU jest natomiast możliwość wystąpienia zjawiska „martwych neuronów”, kiedy to neurony dla wszystkich wejść generują wartość zero i przestają być aktywne w procesie uczenia. Istnieją jednak modyfikacje tej funkcji jak LeakyReLU, które pomagają w minimalizacji skutków tego zjawiska.

Porównanie funkcji i ich pochodnych	
Funkcja Heaviside $H(x)$	Pochodna $H(x)$
	
Funkcja ReLU(x)	Pochodna ReLU(x)
	
Tangens hiperboliczny $\tanh(x)$	Pochodna $\tanh(x)$
	
$\sigma(x)$	Pochodna $\sigma(x)$
	

Nasylenie

W przypadku wielu klasycznych funkcji aktywacji, takich jak funkcja sigmoidalna czy tangens hiperboliczny, występuje zjawisko nasycenia. Jak było już wcześniej wspomniane funkcja tangens hiperboliczny, ale nie tylko ona, dla dużych wartości bezwzględnych argumentu funkcja przyjmuje wartości bliskie swojemu maksimum i minimum, a jej pochodna staje się bliska zero. W procesie uczenia gradientowego prowadzi to do sytuacji, w której sygnał błędu przepływający wstecz przez warstwę zanika. Jest to jedna z głównych przyczyn problemu zanikającego gradientu, który znacząco spowalnia lub wręcz uniemożliwia skuteczne uczenie głębokich modeli. Funkcja ReLU jest odporna na to zjawisko w przypadku wartości dodatnich.

Skośność rozkładu aktywacji

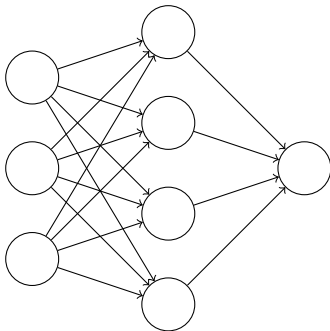
Kolejnym istotnym zjawiskiem jest skośność rozkładu aktywacji (*ang. bias shift*), wynikająca z tego, że niektóre funkcje aktywacji generują wyłącznie wartości dodatnie. Funkcja sigmoidalna jest tego przykładem - jej zakres (0,1) powoduje, że średnia wartość aktywacji jest przesunięta względem zera. W kolejnych warstwach sieci prowadzi to do przesunięcia wartości wejściowych neuronów w kierunku

obszarów nasycenia funkcji aktywacji, co dodatkowo potęguje problem małych gradientów. Funkcje o zakresie symetrycznym względem zera, takie jak $\tanh(x)$, mogą w pewnym stopniu redukować ten efekt, a funkcje typu ReLU całkowicie go unikają dla dodatnich wejść.

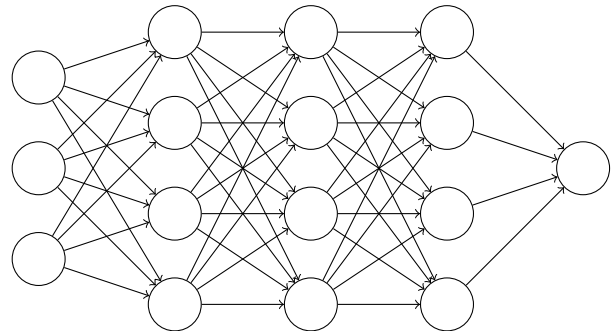
2.2 Architektura sieci neuronowej

W sztucznej sieci neuronowej neurony są zorganizowane w warstwy, gdzie wyjścia neuronów poprzedniej warstwy stają się wejściami neuronów następnej. Typowa struktura to warstwa wejściowa, jedna lub więcej warstw ukrytych oraz warstwa wyjściowa. Zazwyczaj warstwy są ze sobą w pełni połączone (ang. *fully connected*), co oznacza, że każdy neuron danej warstwy jest połączony z każdym neuronem warstwy kolejnej.

Płytka sieć neuronowa



Głęboka sieć neuronowa



Rysunek 1. Porównanie płytkiej i głębokiej sieci neuronowej z wyrównanym położeniem neuronów wyjściowych.

Sieci można klasyfikować m.in. pod względem ich **głębokości**, rozumianej jako liczba warstw ukrytych. Określenie „płytka” (*shallow network*) zazwyczaj odnosi się do sieci z jedną warstwą ukrytą, natomiast sieć „głęboka” (*deep network*) posiada tych warstw wiele. Głębsza architektura umożliwia wielokrotne przekształcenia reprezentacji danych, dzięki czemu kolejne warstwy mogą stopniowo wyodrębniać coraz bardziej złożone cechy – od prostych struktur w warstwach początkowych po wysokopoziomowe abstrakcje w warstwach końcowych.

Z punktu widzenia matematycznego, każdą warstwę można traktować jako funkcję przekształcającą wektor wejściowy w nową reprezentację. Cała sieć jest więc **kompozycją funkcji**:

$$f(x) = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(x) \dots)),$$

gdzie L oznacza liczbę warstw, a $f^{(i)}$ – transformację realizowaną przez i -tą warstwę (obejmującą operację liniową i nieliniową funkcję aktywacji). W przypadku sieci płytkiej z jedną warstwą ukrytą liczba takich zagnieżdżonych kompozycji jest niewielka, co może ograniczać zdolność modelu do efektywnego odwzorowywania złożonych funkcji.

Choć – jak pokaże Twierdzenie Uniwersalnej Aproksymacji – już sieć płytka o odpowiedniej szerokości jest w stanie przybliżyć dowolną funkcję ciągłą na zwartym zbiorze, w praktyce osiągnięcie tego może wymagać bardzo dużej liczby neuronów. Sieci głębokie pozwalają często uzyskać tę samą moc reprezentacyjną przy znacznie mniejszej liczbie jednostek obliczeniowych, rozkładając złożoność zadania na wiele etapów przetwarzania.

Uniwersalna aproksymacja

Twierdzenie uniwersalnej aproksymacji, mówi że odpowiednio „szeroka” (szerokość - liczba neuronów w warstwie) sieć płytka jest w stanie dokonać aproksymacji dowolnej funkcji ciągłej na zwartym zbiorze [Cybenko1989] [HORNICK1989359]. Częściowo tłumaczy to, użyteczność sieci neuronowych. Udowadnia ono, że sztuczne sieci neuronowe mogą prezentować bardzo skomplikowane zależności z wystarczającą ilością neuronów.

Jednak użycie pojedynczej warstwy ukrytej wymaga o wiele większej liczby parametrów niż w przypadku sieci głębokich. Zostało to udowodnione w [RolnickTegmark2017], że całkowita liczba neuronów wymaganych do aproksymacji naturalnych klas wielomianów n zmiennych wielowymiarowych rośnie liniowo $O(n)$ tylko w przypadku głębokich sieci neuronowych, ale rośnie wykładniczo, gdy dozwolona jest tylko jedna warstwa ukryta. Dodatkowo w artykule są prezentowane dowody na to, że przy k warstwach ukrytych wymagana liczba neuronów rośnie wykładniczo nie z n , ale z $n^{\frac{1}{k}}$, co sugeruje, że minimalna liczba warstw wymaganych do praktycznej ekspresji rośnie logarytmicznie z n .

Dzieje się tak, ponieważ głębsze modele wykonują więcej sekwencyjnych przekształceń danych wejściowych, co z punktu widzenia teorii obliczeń odpowiada większej liczbie kroków kompozycji funkcji. $f(x) = f^{(n)}(\dots f^{(3)}(f^{(2)}(f^{(1)}(x))))$, gdzie $f^{(1)}, f^{(2)}, f^{(3)}$ to funkcje realizowane odpowiednio przez kolejne warstwy (wraz z nieliniowymi aktywacjami). W sieci płyckiej mamy tylko jedną taką zagnieżdżoną kompozycję (np. $f(x) = f^{(2)}(f^{(1)}(x))$). [H.N.Mhaskar2020]

Formalnie udowodniono istnienie tzw. hierarchii głębokości: dla pewnych klas funkcji ograniczenie głębokości sieci neuronowej prowadzi do konieczności znaczącego zwiększenia liczby jednostek obliczeniowych. Na przykład, [telgarsky2016benefits] wykazał, że dla dowolnego k można skonstruować funkcję, którą sieć o głębokości k^3 i stałej liczbie neuronów na warstwę jest w stanie efektywnie odwzorować, natomiast każda sieć płytka o głębokości $O(k)$ wymagałaby w przybliżeniu 2^k neuronów. Oznacza to, że wzrost głębokości sieci przekłada się na istotnie większą efektywność reprezentacyjną niż analogiczny wzrost szerokości.

Użyte w tekście symbole O , Ω , to notacje asymptotyczne używane do opisu tempa wzrostu. $O(k)$ oznacza „co najwyżej proporcjonalne do k ”, $\Omega(2^k)$ oznacza dolne ograniczenie

2.3 Techniki uczenia sieci neuronowych

Uczenie sieci neuronowych to proces, podczas którego model dopasowuje swoje parametry w taki sposób, aby minimalizować błąd predykcji. Efektywność tego procesu decyduje o zdolności modelu do generalizacji, czyli poprawnego działania na podstawie danych, których nie widział podczas treningu. W praktyce, uczenie polega na iteracyjnym modyfikowaniu parametrów na podstawie informacji zwrotnej wyrażonej za pomocą funkcji kosztu (straty), której wartość wskazuje, jak bardzo przewidywania modelu odbiegają od pożądanych wyników.

W zależności od dostępności etykiet uczących oraz celu analizy, metody uczenia sieci neuronowych można podzielić na trzy główne kategorie (Bishop, 2006):

- Uczenie nadzorowane, w którym model trenuje na parach danych wejściowych i wynikowych, ucząc się mapowania między wejściem, a oczekiwanym wynikiem.
- Uczenie nienadzorowane ma na celu odkrycie ukrytych zależności w zbiorze danych wejściowych.
- Uczenie ze wzmocnieniem polega na nauce agenta do podejmowania decyzji poprzez interakcje ze środowiskiem, otrzymując nagrody i kary.

Uczenie nadzorowane

Uczenie nadzorowane (*supervised learning*) jest najczęściej stosowanym paradygmatem uczenia sieci neuronowych. Zakłada ono, że dostępny jest zbiór danych uczących składający się z par (x, y) , gdzie x reprezentuje wektor cech opisujących przykład, a y jest przypisaną do niego etykietą lub wartością wyjściową. Celem modelu jest nauczenie się odwzorowania $f: x \rightarrow y$ w taki sposób, aby przewidywane wyjście $\hat{y}$ było jak najbardziej zbliżone do wartości rzeczywistej y [Goodfellow-et-al-2016].

W ramach uczenia nadzorowanego wyróżnia się dwa główne typy zadań:

- **Klasyfikacja** – gdy etykiety y należą do skończonego zbioru klas, a model ma za zadanie przypisać dany przykład do jednej z nich. Przykłady zastosowań obejmują rozpoznawanie obrazów, klasyfikację dokumentów tekstowych czy diagnozowanie chorób na podstawie danych medycznych.
- **Regresja** – gdy etykieta y jest wartością liczbową, a celem jest przewidzenie wartości ciągłej. Do typowych przykładów należą prognozowanie cen, przewidywanie popytu czy estymacja parametrów fizycznych w eksperymentach naukowych.

Proces uczenia w paradygmacie nadzorowanym obejmuje następujące kroki:

1. **Propagacja w przód (forward pass)** – dane wejściowe są przetwarzane przez kolejne warstwy sieci neuronowej, aż do uzyskania prognozy $\hat{y}$.
2. **Obliczenie błędu** – na podstawie funkcji kosztu $L(y, \hat{y})$ mierzona jest różnica między przewidywaniami a wartościami rzeczywistymi.
3. **Propagacja wsteczna (backpropagation)** – przy użyciu reguły łańcuchowej obliczane są gradienty funkcji kosztu względem parametrów modelu [Rumelhart1986].

4. **Aktualizacja wag** – parametry modelu są modyfikowane zgodnie z wybranym algorytmem optymalizacji, np. stochastycznym spadkiem gradientu (SGD) lub Adamem [Kingma2014].

Dzięki swojej uniwersalności, uczenie nadzorowane jest stosowane w szerokim zakresie dziedzin, od rozpoznawania mowy i analizy obrazu, przez systemy rekomendacyjne, aż po prognozowanie finansowe i modelowanie procesów przemysłowych.

Uczenie nienadzorowane

Uczenie nienadzorowane (*unsupervised learning*) obejmuje metody, w których model uczy się wyłącznie na podstawie danych wejściowych x , bez dostępu do odpowiadających im etykiet wynikowych. Celem jest odkrycie wewnętrznej struktury danych, zależności pomiędzy przykładami lub reprezentacji pozwalających na ich bardziej efektywne opisanie [Goodfellow-et-al-2016].

W ramach uczenia nienadzorowanego wyróżnić można kilka głównych typów zadań:

- **Grupowanie (clustering)** – podział zbioru danych na zbiory (klastry) w taki sposób, aby przykłady w tym samym klastrze były do siebie możliwie podobne, a między klastrami występowały wyraźne różnice. Przykładami algorytmów są *k-means*, DBSCAN czy sieci Kohonena (SOM) [Kohonen1990].
- **Redukcja wymiarowości** – przekształcenie danych do przestrzeni o mniejszej liczbie wymiarów przy zachowaniu jak największej ilości istotnej informacji. Popularne metody to analiza głównych składowych (PCA) czy autoenkodery.
- **Uczenie reprezentacji** – poszukiwanie wewnętrznych opisów danych (tzw. reprezentacji ukrytych), które mogą ułatwić dalsze zadania, np. klasyfikację lub generowanie nowych przykładów. Przykładem są modele generatywne, takie jak *Variational Autoencoder* (VAE) czy *Generative Adversarial Networks* (GAN) [GoodfellowGAN2014].

Proces uczenia nienadzorowanego obejmuje zazwyczaj:

1. **Propagację w przód (forward pass)** – przekształcenie danych wejściowych przez kolejne warstwy modelu w celu uzyskania reprezentacji wewnętrznych lub wyników segmentacji.
2. **Obliczenie miary dopasowania** – zamiast klasycznej funkcji straty porównującej prognozy z etykietami, stosuje się miary podobieństwa lub różnicy pomiędzy danymi wejściowymi a ich odwzorowaniem (np. błąd rekonstrukcji).
3. **Propagację wsteczną (backpropagation)** – obliczenie gradientów względem parametrów modelu.
4. **Aktualizację wag** – modyfikację parametrów zgodnie z wybranym algorytmem optymalizacji.

Uczenie nienadzorowane znajduje zastosowanie m.in. w segmentacji obrazów, eksploracji danych, kompresji informacji, wyszukiwaniu podobnych obiektów, detekcji anomalii czy jako etap wstępny w uczeniu nadzorowanym (tzw. *pretraining*).

Funkcje kosztu (straty)

Funkcja kosztu (ang. *loss function*, *cost function*) pełni kluczową rolę w procesie uczenia sieci neuronowych, ponieważ stanowi ilościową miarę różnicy pomiędzy przewidywaniami modelu $\hat{y}$ a wartościami oczekiwanymi y . To właśnie na podstawie wartości funkcji kosztu algorytm optymalizacji oblicza gradienty, które następnie służą do modyfikacji wag modelu w kierunku minimalizacji błędu [Goodfellow-et-al-2016]. Dobór odpowiedniej funkcji straty jest ściśle związany z charakterem problemu i typem danych wyjściowych.

W praktyce można wyróżnić dwie główne grupy funkcji kosztu:

- **Funkcje kosztu dla klasyfikacji** – stosowane, gdy zadaniem modelu jest przypisanie przykładu do jednej z klas:
 - *Entropia krzyżowa (cross-entropy loss)* – powszechnie używana w zadaniach klasyfikacji wieloklasowej i binarnej. Funkcja ta silnie karze błędne przewidywania o wysokim poziomie pewności.
 - *Hinge loss* – często stosowana w klasyfikatorach typu SVM, promuje dużą marginesową separację pomiędzy klasami.
- **Funkcje kosztu dla regresji** – używane, gdy przewidywane są wartości ciągłe:
 - *Mean Squared Error (MSE)* – średnia wartość kwadratu różnic pomiędzy przewidywaniami a wartościami rzeczywistymi; silnie penalizuje duże błędy.
 - *Mean Absolute Error (MAE)* – średnia wartość bezwzględna błędów; mniej wrażliwa na wartości odstające niż MSE.

Dobór funkcji kosztu powinien uwzględniać:

- **Charakter problemu** – np. klasyfikacja binarna vs. wieloklasowa, regresja liniowa vs. nieliniowa.
- **Wrażliwość na wartości odstające** – MSE jest silnie wrażliwe na anomalie, MAE bardziej odporne.
- **Interpretowalność** – niektóre funkcje, takie jak log-loss, mają bezpośrednie powiązanie z teorią prawdopodobieństwa.
- **Stabilność numeryczna** – przy dużej liczbie klas lub ekstremalnych wartościach wyjściowych warto stosować wersje funkcji kosztu zoptymalizowane pod kątem obliczeń numerycznych.

Ostateczny wybór funkcji straty ma istotny wpływ na przebieg procesu uczenia, tempo konwergencji oraz końcową jakość modelu.

Algorytmy optymalizacji

Kluczowym elementem procesu uczenia są Algorytmy optymalizacji. Odpowiadają one za aktualizację parametrów modelu w celu minimalizacji wartości funkcji straty. Optymalizator wykorzystuje z obliczonych gradientów, wyznacza kierunek i wielkość zmian współczynników sieci. [Goodfellow-et-al-2016]

Metoda najszybszego spadku

Podstawową metodą optymalizacji jest spadek gradientu w którym parametry θ są aktualizowane zgodnie z regułą.

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta),$$

gdzie η to współczynnik uczenia (ang. *learning rate*), a $\nabla_{\theta} L(\theta)$ jest gradientem funkcji kosztu względem parametrów. Pomimo prostej zasady działania nie jest ona obecnie najpowszechniej stosowaną metodą uczenia sieci. Jest ona podatna na utknięcie w minimach lokalnych oraz ma niską prędkość zbieżności. Wraz z zbliżaniem do minimum jej prędkość uczenia się drastycznie spada. Aby metoda najszybszego spadku działała poprawnie, należy dobrać poprawną wartość współczynnika prędkości uczenia. Zbyt duża wartość uniemożliwi znalezienie minimum funkcji straty, a zbyt mała zwiększy podatność na utknięcie minimach lokalnych oraz spowolni proces uczenia.

Spadek gradientu z momentem (Momentum)

Metoda *momentum* to usprawnienie klasycznego spadku gradientu, które wprowadza do aktualizacji wag składnik bezwładności (momentum) odpowiednio skalowany przez parametr γ . Pozwala to na użycie większego współczynnika uczenia. Tempo konwergencji znacząco wzrasta, w szczególności w trudnych obszarach funkcji kosztu z dużą ilością minimów lokalnych.

$$1 = \gamma + \eta$$

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} L(\theta_t),$$

$$\theta_{t+1} = \theta_t - v_t.$$

- η — learning rate (wielkość kroku).
- $\gamma \in [0, 1)$ — momentum coefficient, określa stopień zachowania poprzednich gradientów.

Efektom jest filtr wygładzający historię gradientów, co przyspiesza konwergencję i redukuje oscylacje, a także ułatwia przeciskanie przez lokalne minima.

Metoda przyspieszonego gradientu

Metoda przyspieszonego gradientu, nazywana Metodą Nestorova od nazwiska autora, jest dalszym usprawnieniem metody z momentem, w którym gradient jest obliczany nie w bieżącym położeniu parametrów, a w przewidywanym „przyszłym” położeniu modelu. Dzięki temu aktualizacja jest bardziej precyzyjna i stabilna, co pozwala uniknąć nadmiernego przeskoczenia minimum [**Nesterov1983**]

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} L(\theta_t - \gamma v_{t-1}),$$

$$\theta_{t+1} = \theta_t - v_t.$$

RMSProp (Root Mean Square Propagation)

RMSProp to metoda optymalizacji z adaptacyjnym współczynnikiem uczenia, będąca modyfikacją algorytmu AdaGrad. W przeciwieństwie do AdaGrad, który kumuluje wszystkie poprzednie kwadraty gradientów, RMSProp wykorzystuje wykładniczą średnią ruchomą kwadratów gradientów, co pozwala na utrzymanie stabilnych kroków uczenia wraz z postępem treningu [Hinton2012, MediumRMSProp].

Aktualizacje w RMSProp zapisuje się jako:

$$E[g^2]_t = \rho E[g^2]_{t-1} + (1 - \rho) (\nabla_{\theta} L(\theta_t))^2,$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \nabla_{\theta} L(\theta_t),$$

gdzie:

- ρ (typowo ok. 0.9) — współczynnik wygładzania,
- ϵ — mała stała dla stabilności numerycznej,
- η — współczynnik uczenia.

Zalety RMSProp:

- Utrzymuje adaptacyjny krok uczenia bez jego nadmiernej redukcji,
- Poprawia zbieżność w problemach o zmiennych gradientach — często stosowany w RNN.

Wady:

- Wymaga starannego doboru hiperparametrów,
- Może być mniej skuteczny w przypadku ekstremalnie rzadkich gradientów — wtedy lepszy jest Adam.

Adam (Adaptive Moment Estimation)

Adam łączy zalety momentum i RMSProp poprzez adaptacyjne skalowanie kroku uczenia dla każdego parametru na podstawie estymat pierwszego (średnia) i drugiego momentu (wariancja) gradientów. Algorytm jest opisany następująco:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$\theta_{t+1} = \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}},$$

gdzie $g_t = \nabla_{\theta} L(\theta_t)$, β_1 , β_2 są współczynnikami wygładzania (np. 0.9 i 0.999), a ϵ to mała wartość stabilizująca.

Zalety:

- Adaptacyjny krok dla każdego parametru,

- Efektywny obliczeniowo i pamięciowo,
- Działa dobrze przy danych z szumem, gradientach niestabilnych.

Wady:

- Możliwe problemy ze zbieżnością w funkcjach wypukłych,
- Czasem gorsza generalizacja niż klasyczne SGD z momentum,
- Wymaga dostrojenia wielu hiperparametrów.

Wykaz skrótów i symboli

Spis rysunków

1	Porównanie płytkiej i głębokiej sieci neuronowej z wyrównanym położeniem neuronów wyjściowych.	14
---	--	----

Spis tabel